Ben Abbou Nawfal Programmation Web II

Rapport de Projet Web Q3 – 2025-2026 HELBAgent

Riggio Jonathan 2025-2026

Table des matières :

Table des matières

Introduction 3
Fonctionnalités 4
Système d'inscription et de connexion: 4
Extension HELBPlays : 7
Analyse 8
Architecture : 8
Gestion du cooldown : 9
Intégration des IA: 9
Actualisiation automatique avec l'AJAX : 10
Limitations et développement futur 11
Conclusion 12
Lien vers la vidéo 12

Introduction

Le projet HELBAgent a été réalisé dans le cadre du cours de Programmation Web II afin de mettre en pratique le développement web. Comme vu au cours, il s'agit d'une plateforme permettant aux utilisateurs de créer des agents IA personnalisés et d'interagir avec eux via un système de chat en temps réel, tout en gérant son profil et en visualisant des statistiques d'activité.

Le développement a débuté suite aux 10 premières vidéos d'un tutoriel Django du youtubeur Corey Schafer, qui ont servi de point de départ pour la structure générale de l'application, avant d'être adaptées aux besoins du projet HELBAgent. Toutes les fonctionnalités principales demandées ont été implémentées : système d'authentification (inscription, connexion, profils), CRUD complet pour les agents IA, chat conversationnel avec mécanisme de cooldown pour limiter le spam, intégration de deux types de modèles IA (local via Ollama et distant via une API Groq), statistiques graphiques basées sur l'historique des messages, ainsi que l'extension HELBPlays pour une messagerie externe.

Au-delà de l'implémentation technique, le projet demandait également de respecter un ensemble de règles fonctionnelles : séparation entre profils privés (avec statistiques détaillées) et profils publics (lecture seule), restriction des opérations CRUD aux auteurs des agents et refresh automatique du chat sans rechargement de page.

Ce rapport est structuré de la manière suivante : la section "Fonctionnalités" décrit le fonctionnement de l'application du point de vue utilisateur et du point de vue technique. Ensuite, une section "Analyse" expliquant la structure interne de l'application et les choix de conception (avec des diagrammes et leurs explications). Une section "Limitations et développement futur " présentant les limitations, une autocritique et des pistes d'amélioration. Enfin, la conclusion revient sur le travail réalisé et les apprentissages tirés de ce projet.

Fonctionnalités

Système d'inscription et de connexion:

L'application offre un système d'inscription accessible depuis la page d'accueil via un bouton "Register" en haut de page qui redirige vers le formulaire d'inscription. Le formulaire demande un nom d'utilisateur unique (maximum 150 caractères, lettres, chiffres et @/./+/-/_ uniquement), une adresse email valide, et un mot de passe à confirmer avec plusieurs contraintes de sécurité (minimum 8 caractères, pas entièrement numérique, pas trop commun). Après validation, l'utilisateur est redirigé vers la page de connexion où il peut s'identifier avec ses identifiants. Une fois connecté, il est redirigé vers la page d'accueil où la barre de navigation affiche désormais un bouton "Profile" ainsi qu'un bouton "Logout" permettant la déconnexion.

Le système repose sur le module d'authentification Django qui gère automatiquement la validation, le hachage sécurisé des mots de passe et la création des utilisateurs en base de données. Les formulaires utilisent django-crispy-forms configuré avec Bootstrap 4 pour un rendu visuel cohérent. Les fichiers principaux impliqués sont users/forms.py (formulaire d'inscription personnalisé avec champ email), users/views.py (traitement de l'inscription et de la connexion), et les templates register.html et login.html.

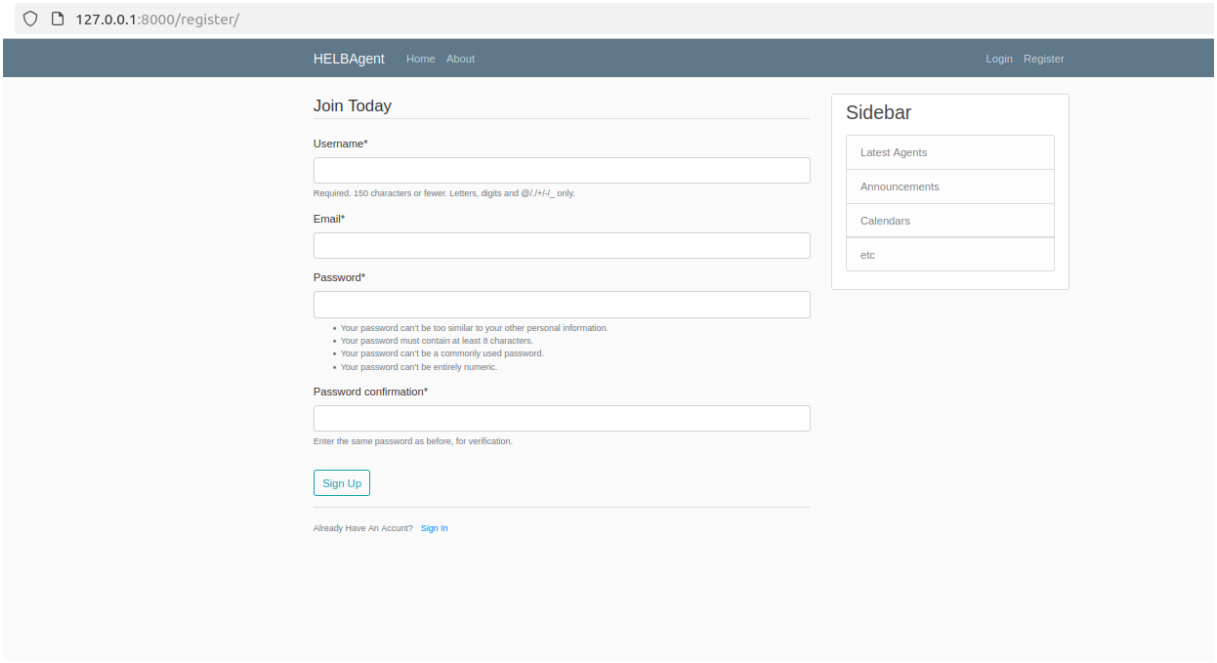


Image 1 : Formulaire d'inscription (Register). Le formulaire "Join Today" permet la création d'un nouveau compte avec validation des champs (nom d'utilisateur, email, mot de passe avec contraintes de sécurité).

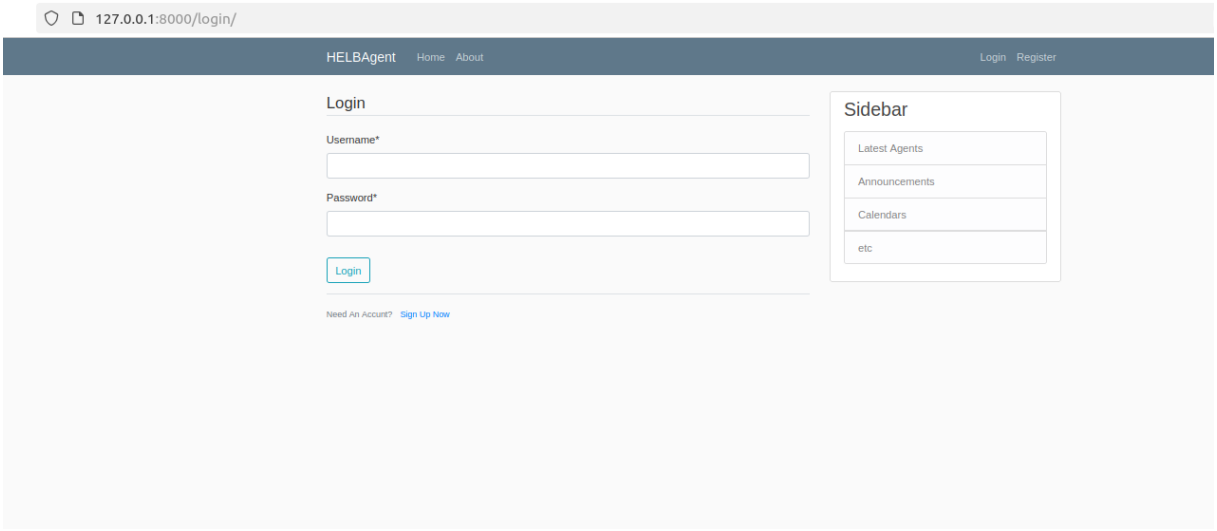


Image 2 : Formulaire de connexion (Login) La page de connexion permet aux utilisateurs existants de s'authentifier avec leur nom d'utilisateur et mot de passe.

Système de profil :

Chaque utilisateur dispose d'un profil unique accessible depuis la barre de navigation. La page de profil privée affiche deux sections distinctes : "Profile Info" avec deux formulaires (un pour modifier le nom d'utilisateur et l'email, un autre pour changer l'avatar avec visualisation de l'image actuelle), et "Activity Statistics" avec des graphiques interactifs montrant la répartition des messages par agent IA et le nombre de messages par jour. Ces statistiques sont calculées dynamiquement à partir de l'historique complet des conversations stocké dans chaque agent. Les profils publics d'autres utilisateurs sont consultables en lecture seule en cliquant sur leurs noms dans les chats, affichant uniquement leurs statistiques publiques sans possibilité de modification.

Le système de profil repose sur une relation un-à-un entre le modèle utilisateur de Django et un modèle Profile personnalisé dans users/models.py contenant les informations de profil (avatar, statistiques). Un signal automatique dans users/signals.py crée le profil lors de l'inscription de chaque nouvel utilisateur. La page de profil gère simultanément deux formulaires distincts (UserUpdateForm et ProfileUpdateForm) dans users/views.py, permettant la modification des informations utilisateur et de l'avatar. L'implémentation utilise Chart.js intégré dans les templates pour générer les graphiques : la vue profile récupère tous les agents créés par l'utilisateur, analyse le champ messages avec des expressions régulières pour extraire les horodatages, compte le nombre de messages par agent et par jour, puis passe ces données aux templates. Les templates profile.html (profil privé) et profile_public.html (profil public) contiennent le code JavaScript initialisant les graphiques Chart.js avec les données fournies, configurant l'affichage selon le type de profil.

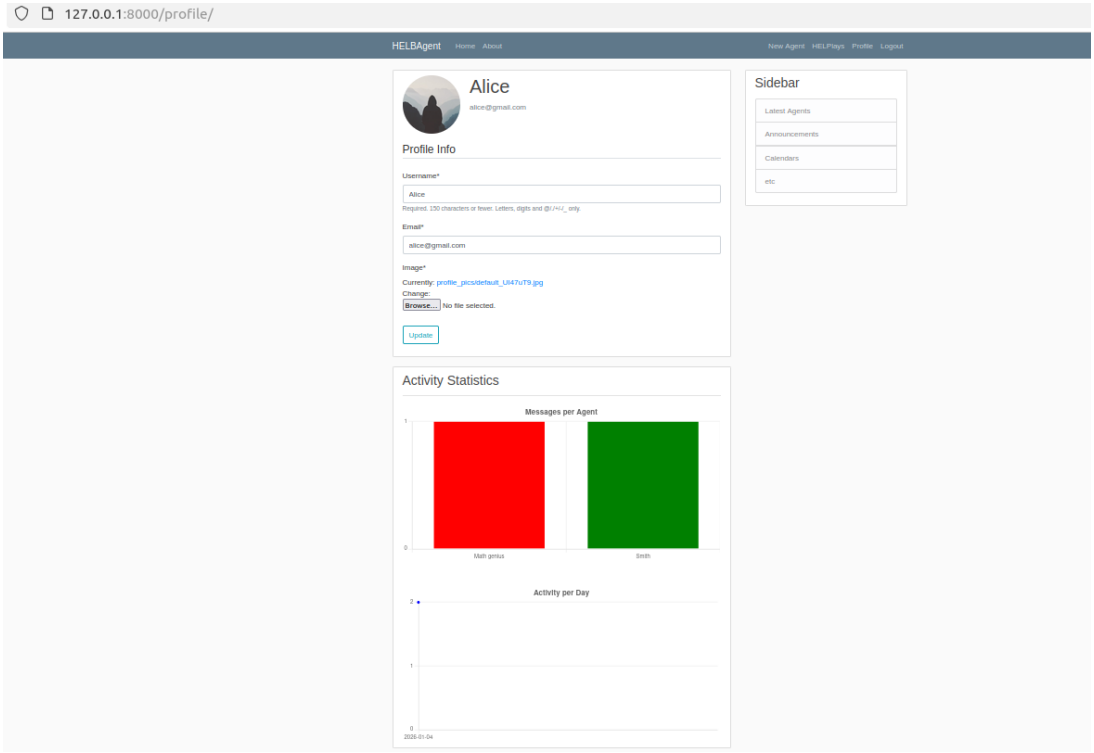


Image 3 : Page de profil privé avec formulaires de modification La page de profil affiche les informations personnalisables de l'utilisateur (nom, email, avatar) et les graphiques montrant la répartition des messages par agent IA et par jour.

CRUD des agents IA :

La page d'accueil liste tous les agents IA créés par la communauté avec leurs informations principales (titre, description, auteur, date de création, avatar). Un utilisateur connecté peut créer un nouvel agent en cliquant sur le bouton "New Agent" et en remplissant un formulaire complet incluant le titre, une description publique, un prompt système (instructions définissant le comportement de l'IA), le choix du modèle IA (Local SmolLM-135M via Ollama ou API Llama-3.3-70b via Groq), un cooldown en secondes (délai minimum entre deux messages), et un avatar personnalisé. Une fois créé, l'agent apparaît dans la liste et possède une page de détail. Seul l'auteur de l'agent voit les boutons "Update" et "Delete" permettant de modifier ses paramètres ou de le supprimer définitivement.

Les opérations CRUD utilisent les vues génériques Django dans blog/views.py : PostListView (liste), PostDetailView (détail), PostCreateView (création), PostUpdateView (modification), et PostDeleteView (suppression). Le modèle Post dans blog/models.py représente un agent IA avec les champs essentiels : titre, description, prompt système, type de modèle IA, cooldown, avatar, historique des messages.

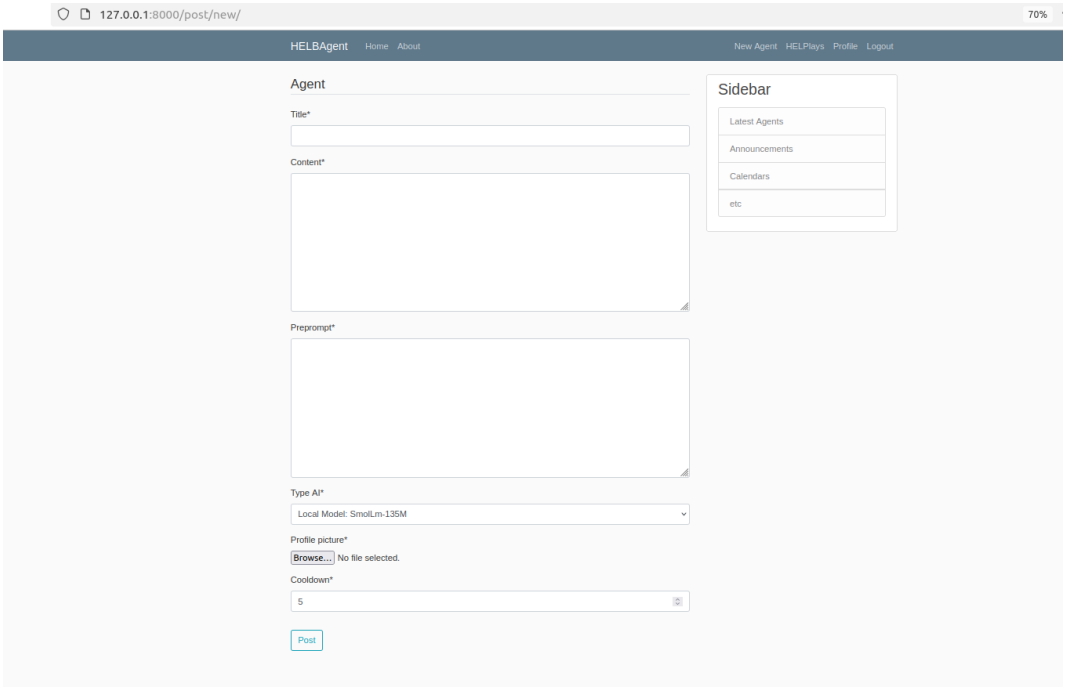


Image 4 : Formulaire de création d'agent Le formulaire "Agent" permet de configurer un nouvel agent avec titre, description (Content), prompt système (Preprompt), choix du modèle IA (Local ou API), une photo de profil (Profile picture) et un cooldown anti-spam.

Extension HELBPlays :

L'extension HELBPlays accessible depuis la barre de navigation propose une messagerie externe autour de l'univers Yu-Gi-Oh, créant un espace de communication partagé entre tous les étudiants de BAC 2 travaillant sur le projet. Cette fonctionnalité indépendante permet aux utilisateurs d'envoyer et de consulter des messages liés aux cartes, stratégies ou discussions Yu-Gi-Oh, créant une communauté parallèle au système de chat avec les agents IA. L'interface propose un formulaire de composition de message et une liste chronologique des messages postés par les étudiants.

L'extension communique avec un serveur externe via des requêtes HTTP dans la fonction helbplays de blog/views.py. Lors de l'envoi d'un message, le nom d'utilisateur, le message et une clé secrète sont encodés en Base64 puis envoyés au serveur distant via une URL construite dynamiquement. Le chat est récupéré depuis le serveur externe et affiché dans le template. Une fonction helbplays_chat_poll permet l'actualisation automatique du chat sans rechargement complet de la page, garantissant une expérience fluide pour suivre les discussions en temps réel.

Analyse

Architecture :

L'ensemble du projet repose sur l'architecture MVT (Model-View-Template) de Django, où chaque composant a une responsabilité clairement définie. Le fichier django_project/urls.py agit comme le routeur principal : il gère les routes globales (admin/, register/, login/, logout/, profile/) et délègue les routes relatives aux agents IA via include('blog.urls') et l'extension HELBPlays. Cette séparation permet de distinguer clairement les responsabilités entre les applications blog (gestion des agents IA), users (gestion des profils utilisateurs), et HELBPlays (messagerie externe).

Le modèle blog/models.py définit la classe Post représentant un agent IA avec ses attributs essentiels: title, content, prompt (instructions système), type_AI (choix local/API), messages (historique complet des conversations stocké en TextField avec séparateur), cooldown, last_activity, profile_picture, et author. Le modèle users/models.py définit Profile en relation 1:1 avec User, contenant l'avatar et les données de profil personnalisées.

Les vues blog/views.py et users/views.py contiennent toute la logique métier : gestion des requêtes HTTP, interactions avec la base de données, appels aux APIs IA (Groq ou Ollama), calculs de statistiques avec expressions régulières, et préparation des données pour l'affichage. Les templates (blog/templates/blog/ et users/templates/users/) affichent les données reçues depuis les vues. L'héritage de templates via {% extends "blog/base.html" %} factorise le HTML commun (navigation, footer, styles) évitant la duplication de code.

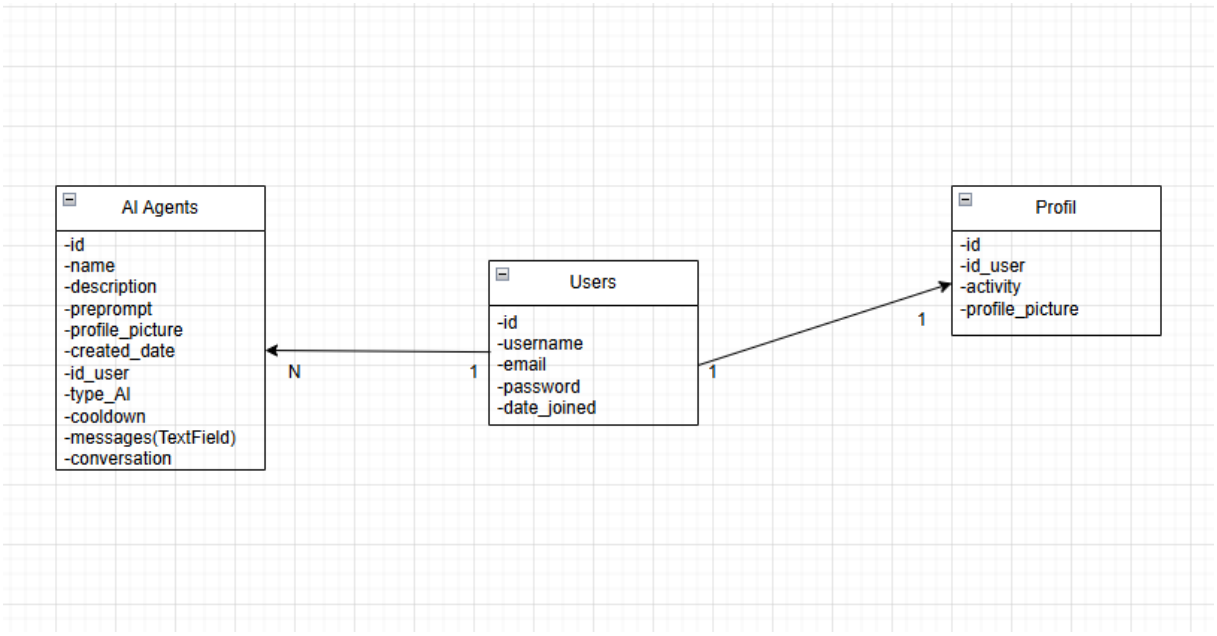


Image 5 : Diagramme entité relation

Ce diagramme d'entité relation illustre les liens principaux du projet HELBAgent et met en évidence les cardinalités entre les trois entités : Users, AI Agents et Profil. Chaque utilisateur peut créer plusieurs agents IA (relation N:1), lui permettant de gérer une collection d'agents personnalisés avec des comportements distincts. Chaque agent est lié à une seule conversation unique stockée dans le champ messages(TextField), garantissant un historique centralisé. Cependant, plusieurs utilisateurs peuvent participer à cette même conversation avec l'agent, permettant ainsi à plusieurs personnes d'échanger avec un agent commun dans un espace partagé. Chaque utilisateur possède également un profil unique (relation 1:1 entre Users et Profil), créé automatiquement lors de l'inscription via un signal Django, isolant les données personnelles et les statistiques d'activité de chaque membre de la plateforme.

Gestion du cooldown :

Le système de cooldown empêche les utilisateurs d'envoyer des messages trop rapidement en imposant un délai configurable entre chaque envoi. Dans la méthode post de PostDetailView dans blog/views.py, le timestamp actuel est récupéré via time.time et comparé avec le dernier timestamp stocké dans la session utilisateur via request.session.get('last_message_time', 0). Si l'écart est inférieur au délai configuré dans self.object.cooldown (champ IntegerField du modèle Post), un message d'avertissement s'affiche (messages.warning(request, f"Calmdown ! The cooldown is {delay_required}s. Wait.")) et l'utilisateur est redirigé sans envoyer le message.

Cette approche utilise les sessions Django plutôt que la base de données pour stocker temporairement le timestamp, évitant des écritures répétées en base et améliorant les performances. Une fois le délai respecté, request.session['last_message_time'] est mis à jour avec le nouveau timestamp et le message est traité normalement. Chaque agent possède son propre cooldown personnalisable (par défaut 5 secondes dans models.py), permettant aux créateurs d'adapter la fréquence d'interaction selon le type d'agent créé.

Intégration des IA:

Le projet intègre deux types de modèles d'IA selon la valeur du champ type_AI du modèle Post. La méthode post de PostDetailView récupère d'abord le prompt système (system_instruction = self.object.prompt) puis traite la requête différemment selon le type configuré.

Lorsque type_AI == 'api', la fonction utilise la bibliothèque groq avec la clé API stockée dans settings.GROQ_API_KEY (configurée dans django_project/settings.py). L'appel se fait via client.chat.completions.create avec le modèle llama-3.3-70b-versatile, en passant deux messages : un message système contenant system_instruction (le prompt personnalisé de l'agent) et un message utilisateur contenant le contenu tapé. La réponse est extraite

via chat_completion.choices[0].message.content et un tag (API): est ajouté pour identifier le type de modèle utilisé. Lorsque type_AI == 'local', la fonction construit un prompt complet (full_prompt = f"System: {system_instruction}. (Be brief).\nUser: {content}\nAssistant:") et exécute la commande système ollama run smollm:135m via subprocess.run avec les paramètres capture_output=True et encoding='utf-8'. La sortie standard (result.stdout.strip) contient la réponse de l'IA locale, et en cas d'erreur, result.stderr est capturé et affiché. Un tag (LOCAL): est ajouté pour différencier ce type de réponse, cette approche nécessitant qu'Ollama soit installé et opérationnel sur le serveur. Les deux réponses sont ensuite ajoutées aes au champ messages du modèle Post avec le séparateur |||, et last_activity est mis à jour avant de sauvegarder l'objet en base de données.

Actualisation automatique avec l'AJAX :

Le chat se rafraîchit automatiquement sans rechargement complet de la page grâce à du JavaScript dans post_detail.html et une vue dédiée post_chat_poll dans blog/views.py. Un setInterval exécute une requête toutes les 2 secondes via fetch("{% url 'post-chat-poll' object.id %}") qui envoie une requête GET vers l'endpoint /post/<int:pk>/chat_poll/ défini dans blog/urls.py. La fonction post_chat_poll(request, pk) récupère l'agent via Post.objects.get(pk=pk), découpe le champ messages avec split(LOG_SEPARATOR), puis génère du HTML pour chaque message : les messages de l'IA (détectés par la présence de (API): ou (LOCAL):) reçoivent la classe CSS alert-secondary, tandis que les messages utilisateurs reçoivent alert-primary text-right. Le tag (API): ou (LOCAL): est supprimé de l'affichage final via replace pour un rendu plus propre, et le HTML généré est retourné via HttpResponse. Côté client, le JavaScript récupère ce HTML (.then(r => r.text())) et l'injecte dans le conteneur #chat-window via innerHTML. Après chaque mise à jour, un script supplémentaire rend les noms d'utilisateur cliquables en les transformant en liens vers leurs profils publics (/profile/${username}/), extrayant le username avant le premier (dans chaque message, et le scroll automatique (chatWindow.scrollTop = chatWindow.scrollHeight) garantit que les derniers messages restent visibles. Le même mécanisme est utilisé pour HELBPlays : helbplays_chat_poll récupère le contenu depuis HELBPLAYS_CHAT_URL (serveur externe) via requests.get et le retourne en tant que texte brut, actualisé toutes les 2 secondes dans helbplays.html.

```python
#fonction pour le refresh auto du chat
def helbplays_chat_poll(request):
    r = requests.get(HELBPLAYS_CHAT_URL, timeout=3)
    return HttpResponse(r.text if r.status_code == 200 else "Error loading chat")
```

Image 6 : Fonction d'actualisation automatique du chat HELBPlays



Image 7 : Script JavaScript d'actualisation automatique du chat HELBPlays

Limitations et développement futur

Le projet présente plusieurs limitations principalement liées aux choix de conception initiaux et au temps disponible pour le développement. L'absence d'un véritable modèle Message en base de données a été une contrainte majeure : toute la conversation est stockée dans un simple champ TextField concaténé avec un séparateur |||, ce qui complique énormément le traitement, les statistiques et l'évolution du système. Avec une table Message séparée contenant les champs auteur, contenu, date, agent et type (IA ou humain), l'architecture aurait été beaucoup plus propre, extensible et maintenable. Le filtrage, le comptage et l'affichage des messages auraient été largement simplifiés grâce aux outils natifs de Django qui permettent d'interroger directement la base de données sans manipulations manuelles de texte.

Ce choix technique impose de gérer manuellement la cohérence du format de texte et rend certaines opérations fragiles, notamment l'extraction des dates via expressions régulières, le découpage manuel des messages et la génération de HTML côté serveur. Un modèle Message dédié aurait permis d'exploiter pleinement les capacités de Django pour créer des statistiques avancées, organiser les conversations et améliorer les performances.

Avec davantage de temps, il aurait été possible de refactoriser complètement le système de stockage des messages autour d'une structure dédiée, rendant le projet plus robuste et évolutif pour de futures améliorations.

Conclusion

Le projet HELBAgent a permis de développer une plateforme web fonctionnelle permettant aux utilisateurs de créer et d'interagir avec des agents IA personnalisés via une interface intuitive construite avec Django. L'application offre des fonctionnalités complètes incluant la gestion d'utilisateurs, la création d'agents configurables, un système de chat en temps réel avec actualisation automatique AJAX, des statistiques visuelles générées dynamiquement avec Chart.js et une extension HELBPlays connectée à un serveur externe. L'intégration de deux types de modèles d'IA (Groq API et Ollama local) montre la flexibilité du système et la capacité à adapter les performances selon les besoins.

L'architecture MVT de Django a facilité l'organisation modulaire du code en séparant clairement les responsabilités entre modèles, vues et templates, permettant une maintenance aisée et des évolutions futures. Les fonctionnalités implémentées répondent aux objectifs initiaux du projet en offrant une expérience utilisateur fluide, des conversations contextuelles avec les agents, et des mécanismes anti-spam via le système de cooldown.

Malgré ces réussites, le projet présente des limitations techniques importantes, notamment l'absence d'un modèle Message dédié qui aurait simplifié la gestion des conversations et amélioré les performances. Les contraintes de temps ont également limité l'implémentation de fonctionnalités avancées et de mécanismes de sécurité robustes. Néanmoins, ce projet constitue une base solide pour de futures améliorations et illustre une maîtrise des concepts fondamentaux du développement web.

Lien vers la vidéo

Voici le lien vers la vidéo YouTube : https://youtu.be/s6i3IHbuY9Q